

Solving Discrete Constraint Satisfaction Problems (CSP)

Agah DUZENLI

March 2026

Problem A: Mathematical Analysis and Search Reduction

Problem A involves the variables A, B, C, D, E , and F , each taking integer values in the domain $\{1, \dots, 120\}$. The constraints are

$$C1 : A = B^2 - C^2 \quad (1)$$

$$C2 : E + F > B \quad (2)$$

$$C3 : D = B^2 - 3A \quad (3)$$

$$C4 : (B - C)^2 = EFC - 1861 \quad (4)$$

$$C5 : C + D + E + F < 120 \quad (5)$$

The goal is to find one valid assignment that satisfies all constraints, while reducing the amount of search as much as possible using algebra.

1. Computing A directly from B and C

Constraint $C1$ gives

$$A = B^2 - C^2.$$

This means that once values for B and C are chosen, the value of A is completely determined. Therefore, there is no need to search over all possible values of A . Instead, the program computes A directly.

2. Computing D directly from B and C

Constraint $C3$ gives

$$D = B^2 - 3A.$$

Since A is already determined by B and C , the value of D is also determined. Substituting $A = B^2 - C^2$ from $C1$ into $C3$ gives

$$D = B^2 - 3(B^2 - C^2).$$

Simplifying:

$$D = B^2 - 3B^2 + 3C^2$$

$$D = 3C^2 - 2B^2.$$

3. Restricting the possible values of C

Because all variables must remain in the domain $\{1, \dots, 120\}$, the value of D must be positive. Using

$$D = 3C^2 - 2B^2,$$

we require

$$3C^2 - 2B^2 > 0.$$

So,

$$3C^2 > 2B^2,$$

$$C^2 > \frac{2}{3}B^2,$$

$$C > B\sqrt{\frac{2}{3}}.$$

Since

$$\sqrt{\frac{2}{3}} \approx 0.8165,$$

we obtain the restriction

$$0.8165B < C.$$

Also, from $A = B^2 - C^2$, if we want A to stay positive, then we need

$$B^2 - C^2 > 0,$$

which implies

$$C < B.$$

Combining both conditions gives

$$0.8165B < C < B.$$

This is useful because instead of trying all values $1, \dots, 120$ for C , the program only tests values in this smaller interval.

4. Rewriting constraint C4 to obtain the product EF

Constraint C4 is

$$(B - C)^2 = EFC - 1861.$$

Adding 1861 to both sides gives

$$(B - C)^2 + 1861 = EFC.$$

So,

$$EFC = (B - C)^2 + 1861.$$

Dividing both sides by C gives

$$EF = \frac{(B - C)^2 + 1861}{C}.$$

Let

$$P = \frac{(B - C)^2 + 1861}{C}.$$

Then

$$EF = P.$$

This means the program does not need to guess E and F independently. Instead, once B and C are known, the product EF is known.

However, since E and F must be integers, the division must be exact. Therefore the program first checks whether

$$((B - C)^2 + 1861) \bmod C = 0.$$

If this is not true, then no integer values of E and F can satisfy C4, so that choice of B and C is discarded immediately.

5. Using C2 and C5 to restrict the sum $E + F$

After finding the product EF , the next step is to restrict the possible values of the sum $E + F$.

From constraint C2,

$$E + F > B.$$

Since $E + F$ is an integer, this means

$$E + F \geq B + 1.$$

So the minimum possible sum is

$$s_{\min} = B + 1.$$

From constraint C5,

$$C + D + E + F < 120.$$

Rearranging gives

$$E + F < 120 - C - D.$$

Since $E + F$ is an integer, this means

$$E + F \leq 119 - C - D.$$

So the maximum possible sum is

$$s_{\max} = 119 - C - D.$$

Therefore, the sum must satisfy

$$B + 1 \leq E + F \leq 119 - C - D.$$

If

$$s_{\min} > s_{\max},$$

then no valid values of E and F exist for that choice of B and C , so the program skips that case.

6. Why the sum must also satisfy $E + F \geq 2\sqrt{EF}$

Suppose

$$E + F = s \quad \text{and} \quad EF = P.$$

For two real numbers with fixed product P , the smallest possible sum occurs when the two numbers are equal. This gives the inequality

$$E + F \geq 2\sqrt{EF}.$$

Since $EF = P$, we get

$$s \geq 2\sqrt{P}.$$

Therefore, before checking possible sums, the program starts at

$$\max(s_{\min}, \lceil 2\sqrt{P} \rceil).$$

This avoids testing sums that could not possibly produce real solutions.

7. Finding E and F using the quadratic formula

Once the product and a candidate sum are known, E and F can be recovered algebraically.

If

$$E + F = s \quad \text{and} \quad EF = P,$$

then E and F are the roots of the quadratic equation

$$x^2 - sx + P = 0.$$

Using the quadratic formula:

$$x = \frac{s \pm \sqrt{s^2 - 4P}}{2}.$$

Therefore,

$$E = \frac{s - \sqrt{s^2 - 4P}}{2}, \quad F = \frac{s + \sqrt{s^2 - 4P}}{2}.$$

The quantity

$$\Delta = s^2 - 4P$$

is called the discriminant.

For E and F to be integers, two things must happen:

1. The discriminant must be nonnegative:

$$\Delta \geq 0.$$

2. The discriminant must be a perfect square, so that $\sqrt{\Delta}$ is an integer.

Also, the numerators must be even so that division by 2 produces integers.

Thus the program checks:

- whether $\Delta \geq 0$,
- whether Δ is a perfect square,
- whether the roots are integers,
- and whether both roots lie in the domain $\{1, \dots, 120\}$.

This is much more efficient than brute-forcing every pair (E, F) .

8. Why B is restricted to divisors of 200

The code also includes the condition that B must divide 200 evenly. This comes from Problem B, specifically constraint $C7$:

$$BEF = HB - 200.$$

Rearranging,

$$HB = BEF + 200,$$

$$H = \frac{BEF + 200}{B}.$$

For H to be an integer, B must divide 200 evenly. Therefore, only values of B such that

$$200 \bmod B = 0$$

are considered.

Strictly speaking, this condition is not required to solve Problem A by itself. However, this restriction is useful if the goal is to find a Problem A solution that can later be extended to Problem B.

9. Summary of the search strategy

Instead of searching all six variables directly, the algorithm uses the following strategy:

1. Choose a value for B such that B divides 200.

2. Choose a value for C satisfying

$$0.8165B < C < B.$$

3. Compute

$$A = B^2 - C^2.$$

4. Compute

$$D = 3C^2 - 2B^2.$$

5. Compute

$$P = \frac{(B - C)^2 + 1861}{C},$$

provided the division is exact.

6. Use constraints $C2$ and $C5$ to find the valid range of

$$s = E + F.$$

7. For each valid sum s , solve the quadratic

$$x^2 - sx + P = 0$$

to obtain E and F .

8. Check that all computed values are integers in the domain and satisfy the constraints.

Algorithm 1 Solve Problem A

```
1: Reset  $nva \leftarrow 0$ 
2: assignment  $\leftarrow$  empty dictionary
3: for  $B \leftarrow 1$  to 120 do
4:   if  $200 \bmod B \neq 0$  then
5:     continue
6:   end if
7:   Assign  $B$ 
8:    $c_{\min} \leftarrow \lfloor 0.8165 \cdot B \rfloor + 1$ 
9:   for  $C \leftarrow c_{\min}$  to  $B - 1$  do
10:    Assign  $C$ 
11:     $A \leftarrow B^2 - C^2$ 
12:    if  $A \notin \{1, \dots, 120\}$  then
13:      continue
14:    end if
15:    Assign  $A$ 
16:     $D \leftarrow B^2 - 3A$ 
17:    if  $D \notin \{1, \dots, 120\}$  then
18:      continue
19:    end if
20:    Assign  $D$ 
21:     $rightSide \leftarrow (B - C)^2 + 1861$ 
22:    if  $rightSide \bmod C \neq 0$  then
23:      continue
24:    end if
25:     $EF\_product \leftarrow rightSide / C$ 
26:     $sum\_min \leftarrow B + 1$ 
27:     $sum\_max \leftarrow 119 - C - D$ 
28:    if  $sum\_min > sum\_max$  then
29:      continue
30:    end if
31:     $real\_sum\_min \leftarrow \lceil 2\sqrt{EF\_product} \rceil$ 
32:     $sum\_start \leftarrow \max(sum\_min, real\_sum\_min)$ 
33:    for  $s \leftarrow sum\_start$  to  $sum\_max$  do
34:       $disc \leftarrow s^2 - 4 \cdot EF\_product$ 
35:      if  $disc < 0$  then
36:        continue
37:      end if
38:       $root \leftarrow \lfloor \sqrt{disc} \rfloor$ 
39:      if  $root \cdot root \neq disc$  then
40:        continue
41:      end if
42:      if  $(s - root) \bmod 2 \neq 0$  then
43:        continue
44:      end if
45:       $E \leftarrow (s - root) / 2$ 
46:       $F \leftarrow (s + root) / 2$ 
47:      if  $E \notin \{1, \dots, 120\}$  or  $F \notin \{1, \dots, 120\}$ 
48:      then
49:        continue
50:      end if
51:      Assign  $E$ 
52:      Assign  $F$ 
53:      if  $E + F \leq B$  then
54:        continue
55:      end if
56:      if  $C + D + E + F \geq 120$  then
57:        continue
58:      end if
59:      return ( $true$ ,  $assignment$ ,  $nva$ )
60:    end for
61:  end for
```

Problem B: Mathematical Analysis and Search Reduction

Problem B extends a valid solution of Problem A by introducing the new variables G, H, I , and J . The new constraints are

$$C6: (G + I)^3 = (H - A - 1)^2 \quad (6)$$

$$C7: B \cdot E \cdot F = H \cdot B - 200 \quad (7)$$

$$C8: (C + I)^2 = B \cdot E \cdot (G + 1) \quad (8)$$

$$C9: G + I < E \quad (9)$$

$$C10: D + H > 180 \quad (10)$$

$$C11: J < H - C - G \quad (11)$$

$$C12: J > B \cdot G + D + E + G \quad (12)$$

In this part, the values A, B, C, D, E , and F are already known from a valid solution of Problem A. The goal is to determine whether this partial solution can be extended by finding valid values for G, H, I , and J .

1. Computing H directly from $C7$

Constraint $C7$ is

$$B \cdot E \cdot F = H \cdot B - 200.$$

Rearranging gives

$$H \cdot B = B \cdot E \cdot F + 200$$

and therefore

$$H = \frac{B \cdot E \cdot F + 200}{B}.$$

Since B, E , and F are already known, the value of H is completely determined.

$$H = EF + \frac{200}{B}.$$

This works because B has already been restricted so that it divides 200 evenly. After computing H , the algorithm checks whether H is still in the allowed domain $\{1, \dots, 120\}$.

2. Using $C10$ to discard impossible cases early

Constraint $C10$ is

$$D + H > 180.$$

Since both D and H are already known at this point, this is a direct test. If

$$D + H \leq 180,$$

then the current partial solution can never satisfy Problem B, so it is rejected immediately.

3. Restricting $G + I$ using $C6$

Constraint $C6$ is

$$(G + I)^3 = (H - A - 1)^2.$$

Let

$$\delta = H - A - 1.$$

Then $C6$ becomes

$$(G + I)^3 = \delta^2.$$

The left-hand side is a cube, while the right-hand side is a square. Therefore, the common value must be both a perfect cube and a perfect square, which means it must be a perfect sixth power.

Suppose

$$G + I = m^2.$$

Then

$$(G + I)^3 = (m^2)^3 = m^6.$$

So we must also have

$$\delta^2 = m^6,$$

which implies

$$|\delta| = m^3.$$

Within the allowed domain, this gives only a few possible values:

$$m = 2 \Rightarrow G + I = 4, \quad |\delta| = 8$$

$$m = 3 \Rightarrow G + I = 9, \quad |\delta| = 27$$

$$m = 4 \Rightarrow G + I = 16, \quad |\delta| = 64.$$

Thus the only possible values for $G + I$ are

$$G + I \in \{4, 9, 16\}.$$

4. Using $C9$ to eliminate impossible sums

Constraint $C9$ is

$$G + I < E.$$

Once the value of $G + I$ has been determined from $C6$, this condition can be checked immediately. If

$$G + I \geq E,$$

then no valid extension is possible and the algorithm stops exploring that case.

5. Computing I once G is chosen

After the exact value of $G + I$ is known, the algorithm only needs to choose a value for G . Then I is determined by

$$I = (G + I) - G.$$

So the algorithm does not independently search both G and I . It only loops over possible values of G , and computes I directly.

This reduces the search further.

6. Using $C8$ to test valid pairs (G, I)

Constraint $C8$ is

$$(C + I)^2 = B \cdot E \cdot (G + 1).$$

Since B, C , and E are already known, and I is computed from G , this becomes a direct test. Any pair (G, I) that does not satisfy this equation is rejected immediately.

7. Restricting the search range for J using $C11$ and $C12$

The variable J is constrained by two inequalities:

$$C11: \quad J < H - C - G$$

and

$$C12: \quad J > B \cdot G + D + E + G.$$

Since all quantities on the right-hand side are known once G is fixed, these two inequalities give a very small interval in which J can lie. Because J is an integer, the bounds become

$$J_{\min} = B \cdot G + D + E + G + 1$$

and

$$J_{\max} = H - C - G - 1.$$

Therefore, instead of searching all values from 1 to 120, the algorithm only checks

$$J_{\min} \leq J \leq J_{\max}.$$

This is a significant reduction in search effort.

8. Final verification of all constraints

Even though the previous steps already enforce the constraints, the code performs one final combined check of all Problem B constraints:

$$\begin{aligned}
(G + I)^3 &= (H - A - 1)^2 \\
B \cdot E \cdot F &= H \cdot B - 200 \\
(C + I)^2 &= B \cdot E \cdot (G + 1) \\
G + I &< E \\
D + H &> 180 \\
J &< H - C - G \\
J &> B \cdot G + D + E + G
\end{aligned}$$

If all are satisfied, then the Problem A solution has been successfully extended to a valid Problem B solution.

9. Summary of the Problem B strategy

Instead of brute-forcing all four new variables, the algorithm uses algebraic reductions:

1. Compute H directly from $C7$.
2. Check $C10$ immediately.
3. Use $C6$ to determine the exact required value of $G + I$.
4. Use $C9$ to discard impossible cases.
5. Loop over possible G values and compute

$$I = (G + I) - G.$$

6. Check $C8$.
7. Use $C11$ and $C12$ to restrict the possible values of J .
8. Return the first valid extension found.

This makes Problem B much more efficient than searching all possible values of G, H, I , and J independently.

Algorithm 2 Extend a Problem A solution to Problem B

Require: A valid solution of Problem A containing A, B, C, D, E, F

Ensure: A valid solution of Problem B, if one exists

- 1: Read A, B, C, D, E, F from the Problem A solution
- 2: Compute

$$H \leftarrow EF + \frac{200}{B}$$

- 3: **if** $H \notin \{1, \dots, 120\}$ **then**
- 4: **return** no solution
- 5: **end if**

6: Increment nva

- 7: **if** $D + H \leq 180$ **then**

8: **return** no solution

9: **end if**

10: $\delta \leftarrow H - A - 1$

11: Create map:

$$\{8 \mapsto 4, 27 \mapsto 9, 64 \mapsto 16\}$$

- 12: **if** $|\delta| \notin \{8, 27, 64\}$ **then**

13: **return** no solution

14: **end if**

15: $GI \leftarrow$ mapped value of $|\delta|$

- 16: **if** $GI \geq E$ **then**

17: **return** no solution

18: **end if**

- 19: **for** $G \leftarrow 1$ to $GI - 1$ **do**

20: $I \leftarrow GI - G$

- 21: **if** $G \notin \{1, \dots, 120\}$ or $I \notin \{1, \dots, 120\}$ **then**

22: **continue**

23: **end if**

24: Increment nva by 2

- 25: **if** $(C + I)^2 \neq B \cdot E \cdot (G + 1)$ **then**

26: **continue**

27: **end if**

28: $J_{\min} \leftarrow B \cdot G + D + E + G + 1$

29: $J_{\max} \leftarrow H - C - G - 1$

- 30: **for** $J \leftarrow J_{\min}$ to $J_{\max}$ **do**

- 31: **if** $J \notin \{1, \dots, 120\}$ **then**

32: **continue**

33: **end if**

34: Increment nva

- 35: **if** $(G + I)^3 = (H - A - 1)^2$ **then**

36: **if** $B \cdot E \cdot F = H \cdot B - 200$ **then**

37: **if** $(C + I)^2 = B \cdot E \cdot (G + 1)$ **then**

38: **if** $G + I < E$ **then**

39: **if** $D + H > 180$ **then**

40: **if** $J < H - C - G$ **then**

41: **if** $J > B \cdot G + D + E + G$

42: **then**

43: Return Problem B solution with G, H, I, J

44: **end if**

45: **end if**

46: **end if**

47: **end if**

48: **end if**

49: **end if**

50: **end for**

Problem C: Mathematical Analysis and Constraint Derivation

Problem C extends a valid solution of Problem B by introducing three additional variables:

$$K, L, M$$

All variables must remain within the domain

$$\{1, \dots, 120\}.$$

The new constraints are

$$C13 : KLM = B(B + 5) \quad (13)$$

$$C14 : F^3 = K^2M^2 \cdot 10 + 331 \quad (14)$$

$$C15 : HM^2 = JK - 20 \quad (15)$$

$$C16 : J + L = IL \quad (16)$$

$$C17 : A + D + M = B(F - 2) \quad (17)$$

Since the variables A, B, D, F, H, I , and J are already determined by the solution of Problem B, the goal of Problem C is to determine whether valid values of K, L , and M exist that satisfy these constraints.

Instead of searching over all possible values of K, L , and M , algebraic manipulation allows these variables to be computed directly.

1. Computing M using constraint $C17$

Constraint $C17$ is

$$A + D + M = B(F - 2).$$

Solving for M gives

$$M = B(F - 2) - A - D.$$

Since A, B, D , and F are already known, the value of M is completely determined. Therefore the algorithm does not need to search over possible values of M .

After computing M , the algorithm checks whether

$$1 \leq M \leq 120.$$

If this condition is not satisfied, the current Problem B solution cannot be extended to Problem C.

2. Computing K using constraint $C15$

Constraint $C15$ is

$$HM^2 = JK - 20.$$

Rearranging gives

$$JK = HM^2 + 20.$$

Solving for K :

$$K = \frac{HM^2 + 20}{J}.$$

Thus K can be computed directly once M is known.

However, because K must be an integer, the numerator must be divisible by J . Therefore the algorithm checks whether

$$(HM^2 + 20) \bmod J = 0.$$

If this condition fails, then no valid integer value of K exists for the current solution.

After computing K , the algorithm again verifies that

$$1 \leq K \leq 120.$$

3. Computing L using constraint $C16$

Constraint $C16$ is

$$J + L = IL.$$

Rearranging gives

$$J = IL - L.$$

Factoring out L :

$$J = L(I - 1).$$

Solving for L :

$$L = \frac{J}{I - 1}.$$

Thus L is also determined algebraically once I and J are known.

To ensure L is an integer, the algorithm verifies

$$J \bmod (I - 1) = 0.$$

Additionally, the case $I = 1$ must be avoided because it would cause division by zero.

After computing L , the domain condition

$$1 \leq L \leq 120$$

is checked.

4. Verifying the remaining constraints

After computing M, K , and L , two constraints remain to be verified.

First, constraint $C13$:

$$KLM = B(B + 5).$$

This checks whether the product of the newly computed variables matches the value determined by B .

Second, constraint $C14$:

$$F^3 = K^2M^2 \cdot 10 + 331.$$

This constraint links the newly computed variables to F , which was determined earlier in Problem A.

If both constraints are satisfied, then the Problem B solution has been successfully extended to a valid Problem C solution.

5. Summary of the Problem C strategy

Instead of searching three new variables, the algorithm uses algebra to compute them directly:

1. Compute M using constraint $C17$.
2. Compute K using constraint $C15$.
3. Compute L using constraint $C16$.
4. Verify the remaining constraints $C13$ and $C14$.

This eliminates almost all search in Problem C, making the solution extremely efficient.

Algorithm 3 Extend a Problem B solution to Problem C

Require: A valid solution of Problem B containing A, B, D, F, H, I, J

- 1: Compute $M \leftarrow B(F - 2) - A - D$
 - 2: **if** $M \notin \{1, \dots, 120\}$ **then**
 - 3: **return** no solution
 - 4: **end if**
 - 5: Increment nva
 - 6: Compute $k_num \leftarrow HM^2 + 20$
 - 7: **if** $k_num \bmod J \neq 0$ **then**
 - 8: **return** no solution
 - 9: **end if**
 - 10: $K \leftarrow k_num/J$
 - 11: **if** $K \notin \{1, \dots, 120\}$ **then**
 - 12: **return** no solution
 - 13: **end if**
 - 14: Increment nva
 - 15: **if** $I \leq 1$ **then**
 - 16: **return** no solution
 - 17: **end if**
 - 18: **if** $J \bmod (I - 1) \neq 0$ **then**
 - 19: **return** no solution
 - 20: **end if**
 - 21: $L \leftarrow J/(I - 1)$
 - 22: **if** $L \notin \{1, \dots, 120\}$ **then**
 - 23: **return** no solution
 - 24: **end if**
 - 25: Increment nva
 - 26: **if** $KLM \neq B(B + 5)$ **then**
 - 27: **return** no solution
 - 28: **end if**
 - 29: **if** $F^3 \neq K^2M^2 \cdot 10 + 331$ **then**
 - 30: **return** no solution
 - 31: **end if**
 - 32: Return solution including K, L, M
-

Overall Solver Structure

After defining the procedures for Problem A, Problem B, and Problem C, the program uses three additional components:

1. a general solver function `solve_csp(problem)`,
2. a verification function `verify(problem, sol)`,
3. and a main driver function `main()`.

These parts control the hierarchical search, check correctness of the returned solution, and write the final results to a CSV file.

1. The `solve_csp(problem)` function

The purpose of `solve_csp(problem)` is to provide one unified interface for solving Problem A, Problem B, or Problem C.

The input is a letter:

A, B, or C.

The function first converts the input to uppercase so that lowercase input such as `a`, `b`, or `c` will still work correctly.

Then the counter

nva

is reset to zero. This ensures that each run starts with a fresh count of the number of variable assignments.

Hierarchical solving process

The search is organized hierarchically:

- Problem A is solved first.
- Problem B is attempted only from valid solutions of Problem A.
- Problem C is attempted only from valid solutions of Problem B.

This matches the structure of the assignment, since Problems B and C both contain all earlier constraints.

2. The `verify(problem, sol)` function

The purpose of `verify(problem, sol)` is to confirm that the returned solution really satisfies all required constraints.

This is a safety check. Even if the search procedure appears correct, verification ensures that the final answer is mathematically valid.

3. The `main()` function

The function `main()` is the driver of the whole program. It runs the solver, verifies the results, stores them, and writes them to a CSV file.

4. Summary

The final structure of the program can be summarized as follows:

1. `problem_A()`, `problem_B()`, and `problem_C()` generate hierarchical solutions.
2. `solve_csp(problem)` chooses which level of the hierarchy to solve.
3. `verify(problem, sol)` checks that the returned solution satisfies all required constraints.

4. `main()` runs the full process for Problems A, B, and C and writes the results to a CSV file.

Together, these functions form the complete CSP solver.